MSCIT SEM 1 - MONGODB

Chapter-5

Handling Files with GridFS

- **Handling Files with GridFS**
- What is Grid?
- Storing files in GridFS
- Serving files from GridFS
- Reading files in chunks
- **Database Management**
- Database Administration
- Optimization
- Replication
- shade

Q - 1 Explain Grid.

A Grid class is representation of a simple grid interface.

Gridfs is utility for storing and retrieving files from the database.

Syntax: - class Grid (db, fs name)

This Grid class have 2 arguments

1 db(db):- a database instance to interact with

2 [FS name](string):- it is optional argument that provide root collection for gridFS.

This class return a grid as a output.

Grid support different function to perform different operation like.

(1) put() :- it puts binary data to the grid.

(2) get():- It gets binary data to grid.

(3) delete():- it delete the file from the grid.

(1) put():-

It is used to puts the binary data to the grid.

Syntax:- put(data[,optional],callback)

Hear in the argument

Data is buffer data with binary data.

[optional] argument include different options for file like, _id, root, context type, chunk size or metadata.

With the callback will be called after this method is executed.

(2) get():-

This method is used to get binary data to the grid.

Syntax:- get(id, callback)

Hear the 2 arguments have

Id is used for any file.

With the callback will be called after this method is executed.

(3) delete():-

It is used to delete the file from the grid.

Syntax:- delete(id, callback)

Id is used for any file.

With the callback will be called after this method is executed.

what is gride fs?

- GridFS is the mongodb specification for storing and retrieving large files such as images, audio files, video, sound recorder etc.

- It is a file system to store the files but it's data is stored within mongodb collection.

- GridFS has a capability to store files even greater then it's document size.

- GridFS by default use two collection.

- FS.Files and FS.chunks to storing the data in grid files system.

- GridFS divided a file into chunks and store each chunks and store each chunks of data in a separate document with maximum size 255kb.

- The GridFS is used for storing the files larger than 16 MB.

- You can use GridFS to store as many files as needed in a directory.

- The GridFS is also helpful to access a portion of large file without loading the whole file into memory.

- GridFS is also used when you want to keep your file and metadata automatically display number of system.

- GridFS support two collection to store file metadata and the chunks

- Each chunks is identified by its unique object id field.

- Hear the FS.Files work as a parent document and the FS chunks document can link with files_id field to its parent.

Advantage:-

1. GridFS can simply your stack.

2. GridFS can used instead of a separate tool for file storage.

3. GridFS is helpful and easy in fail over and scale out situation.

4. GridFS support large number of files to be store in the same directory.

Disadvantage:-

1. The performance is slower for accessing the files.

2. You can only modify the document by deleting them and resaving again.

3. It cannot lock all the chunks in a files at the same time.

GridFS is not used in 2 situation:-

1. If the files are smaller than 16 MB.

2. If you need to update content of the entire file automatically.

 Use GridFS:-

- To perform the GridFS operation for storing and retrieving files you have to use.

1. A mongodb driver

2. Mongo files command line tools.

 GridFS collection:-

- the GridFS store files in 2 collection

1. the chunks collection(binary chunks)

2. the file collection (metadata)

- GridFS place the collection in a common bucket which by default start with FS name.

- Ex:- FS.files & FS.chunks

When to Use GridFS

In MongoDB, use GridFS for storing files larger than 16 MB.

In some situations, storing large files may be more efficient in a MongoDB database than on a system-level file system.

 If your file system limits the number of files in a directory, you can use GridFS to store as many files as needed.

 When you want to access information from portions of large files without having to load whole files into memory, you can use GridFS to recall sections of files without reading the entire file into memory.

 When you want to keep your files and metadata automatically synced and deployed across a number of systems and facilities, you can use GridFS. When using geographically distributed replica sets, MongoDB can distribute files and their metadata automatically to a number of mongod instances and facilities.

Do not use GridFS if you need to update the content of the entire file atomically. As an alternative, you can store multiple versions of each file and specify the current version of the file in the metadata. You can update the metadata field that indicates "latest" status in an atomic update after uploading the new version of the file, and later remove previous versions if needed.

Furthermore, if your files are all smaller then 16 MB BSON Document Size limit, consider storing the file manually within a single document instead of using GridFS. You may use the BinData data type to store the binary data.

To store and retrieve files using *GridFS*, use either of the following:

Use GridFS files

- A MongoDB driver.
- The mongofiles command-line tool.

Use GridFS Collections

GridFS stores files in two collections:

- chunks stores the binary chunks.
- files stores the file's metadata.

GridFS places the collections in a common bucket by prefixing each with the bucket name. By default, GridFS uses two collections with a bucket named fs:

- fs.files
- fs.chunks

You can choose a different bucket name, as well as create multiple buckets in a single database. The full collection name, which includes the bucket name, is subject to the namespace length limit.

The chunks Collection

Each document in the chunks collection represents a distinct chunk of a file as represented in *GridFS*. Documents in this collection have the following form:

{

"_id" : <ObjectId>,

"files_id" : <ObjectId>,

"n" : <num>,

"data" : <binary>

}

A document from the chunks collection contains the following fields:

chunks._id

The unique *ObjectId* of the chunk.

chunks.files_id

The _id of the "parent" document, as specified in the files collection.

chunks.n

The sequence number of the chunk. GridFS numbers all chunks, starting with 0.

chunks.data

The chunk's payload as a *BSON* Binary type

The files Collection

Each document in the files collection represents a file in *GridFS*. Consider a document in the files collection, which has the following form:

{

"_id" : <ObjectId>,

"length" : <num>,

"chunkSize" : <num>,

"uploadDate" : <timestamp>,

"md5" : <hash>,

"filename" : <string>,

"contentType" : <string>,

"aliases" : <string array>,

"metadata" : <dataObject>,

}

Documents in the files collection contain some or all of the following fields:

files._id

The unique identifier for this document. The _id is of the data type you chose for the original document. The default type for MongoDB documents is *BSON ObjectId*.

files.length

The size of the document in bytes.

files.chunkSize

The size of each chunk in **bytes**. GridFS divides the document into chunks of size chunkSize, except for the last, which is only as large as needed. The default size is 255 kilobytes (kB).

Changed in version 2.4.10: The default chunk size changed from 256 kB to 255 kB.

files.uploadDate

The date the document was first stored by GridFS. This value has the Date type.

files.md5

An MD5 hash of the complete file returned by the *filemd5* command. This value has the String type.

files.filename

Optional. A human-readable name for the GridFS file.

files.contentType

Optional. A valid MIME type for the GridFS file.

files.aliases

Optional. An array of alias strings.

files.metadata

Optional. Any additional information you want to store.

Applications may create additional arbitrary fields.

MongoDB GridFS Indexes

GridFS uses indexes on each of the chunks and files collections for efficiency. *Drivers* that confirm to the GridFS specification automatically create these indexes for convenience. You can also create any additional indexes as desired to suit your application's needs.

There are two types of Indexes.

1. The chunks Index

2. The file index

The chunks Index

GridFS uses a *unique*, *compound* index on the chunks collection using the files_id and n fields. This allows for efficient retrieval of chunks, as demonstrated in the following example:

db.fs.chunks.find({ files_id: myFileID }).sort({ n: 1 })

Drivers that conform to the GridFS specification will automatically ensure that this index exists before read and write operations.

If this index does not exist, you can issue the following operation to create it using the mongo shell:

db.fs.chunks.createIndex({ files_id: 1, n: 1 }, { unique: true });

The files Index

GridFS uses an *index* on the files collection using the filename and uploadDate fields. This index allows for efficient retrieval of files, as shown in this example:

db.fs.files.find({ filename: myFileName }).sort({ uploadDate: 1 })

Drivers that conform to the GridFS specification will automatically ensure that this index exists before read and write operations.

If this index does not exist, you can issue the following operation to create it using the mongo shell:

db.fs.files.createIndex({ filename: 1, uploadDate: 1 });

[1]	The use of the term *chunks* in the context of GridFS is not related to the use of the term *chunks* in the context of sharding.

Explain gridFS command

- gridFS support all the command of grid class which are as under.

1. List:- It display the list of all files.

2. Search:- It search al files you can added file name as sub string for the search operation.

3. Put:- It added a file with name.

4. Get:- It used to get a file with file name.

5. Delete :- It delete all files or file with file name.

Q4 Explain storing files, reading files and serving file with gridFS

- Mongofiles command is used to perform different operation with gridFS like storing and retrieving files from local system to gridFS.

- Mongofiles is a separate command so you have to perform the mongofiles command by opening command prompt.

- Hear you have a command prompt for server(mongod), one command prompt for client(mongo) and one command prompt for mongofiles.

- Mongofiles include following commands.

--help:-

- Ex:- mongofile --help

- o/p :-return all supported command with syntax.

--version:-

- Ex:-this command is used to return the version of mongofiles.

List:-

- It will display the file list of mongo connected to local host.

- It also display the size of file.

- Sy:- mongofiles –d<dbname>list

- Ex:- mongofiles –d test list.

Put:-

- If you want to add or store the file into grideFS then put command is used.

- You can store different types of files like text files, image file, mp3, etc…

- First you have to put some text file for example “demo.txt” into bin directory now if you want to add this file into grideFS then follow the following syntax.

- Sy:- mongofile –d<db name> put<file name>

- Ex:- mongofile –d test put demo.txt.

- o/p :- add file demo.txt connected to localhost.

mongofiles.exe -d gridfs put song.mp3

Find:-

- Display files with the help of find command.

- If you want to display FS.files in the client windows then put this command

- db.fs.files. find().pretty() in the client dos screen.

- o/p:- -id, chunks size, upload data, length, md5 and filename will display as a output.

- If u want to display the information about files from chunks collection then you have to write the command in the client dos screen like

- db.fs.chunks.find().pretty()

- It display the output like _id, file_id, n and data.

Delete:-

- If you want to delete a file of grideFS then you have to use the delete command in the mongofiles command prompt.

- Sy:- mongofiles –d<dbname> delete<file name>

- Ex:- mongofile –d SMGC delete demo.txt

- o/p:- connected to localhost successfully delete all instance of “demo.txt” from grideFS.

db.fs.files.remove({})

db.fs.chunks.remove({})

db.repairDatabase() (cleans out the memory

Get command & mongofile:-

- The get command is used to retrieve the file from grideFS to the local system.

- It does not display the content of file directly.

- Sy:- mongofile –d<db name> get<file name>

- Ex:- mongofile –d hns get demo.txt

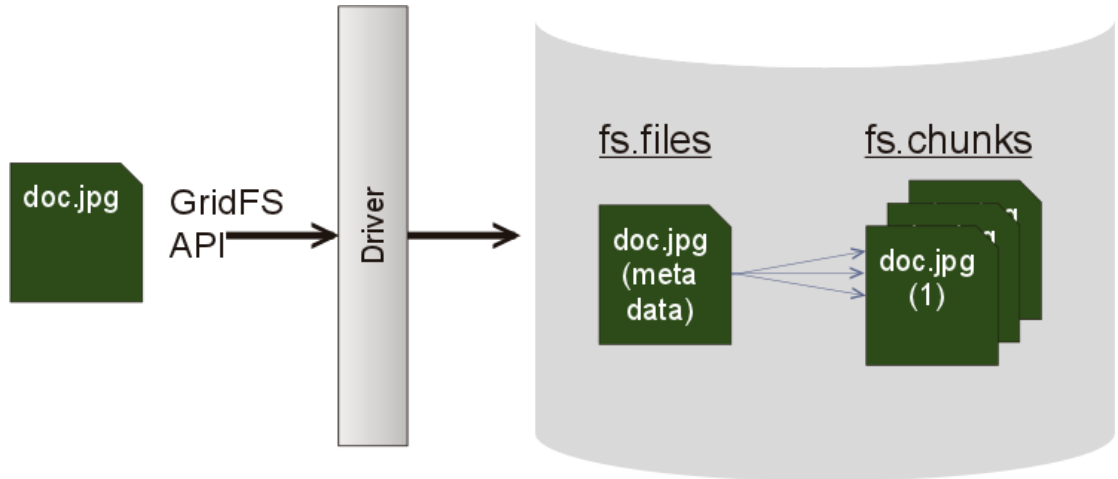
```
<?php
$c=new mongoclient();
$db=$c->hns;
$gets=$db->getgrideFS();
Echo $gets->findOne(array(“filename”=>”demo.txt”))->getBytes(); ?>
```

Note:- The mongofiles utility makes it possible to manipulate files stored in your MongoDB instance in GridFS objects from the command line. It is particularly useful as it provides an interface between objects stored in your file system and GridFS.

Sharding GridFS There are two collections to consider with gridfs - files and chunks. If you need to shard a GridFS data store, use the chunks collection setting { files_id : 1, n : 1 } or { files_id : 1 } as the shard key index. files_id is an objectid. When you use sharding concept then the files collection use the _id field, possibly in combination with an application field.

You cannot use Hashed Sharding when sharding the chunks collection. The files collection is small and only contains metadata.

Leaving files unsharded allows all the file metadata documents to live on the primary shard.



Possible commands include:

list - list all files; 'filename' is an optional prefix which listed filenames must begin with...

search - search all files; 'filename' is a substring which listed filenames must contain

put - add a file with filename 'filename'

get - get a file with filename 'filename'

get_id - get a file with the given '_id'

delete - delete all files with filename 'filename'

delete_id - delete a file with the given '_id'

(1) C:\MongoDB>mongofiles --help

Output:-

general options:

/help print usage

/version print the tool version and exit

verbosity options:

/v, /verbose more detailed log output (include multiple

times for more verbosity, e.g. -vvvvv)

/quiet hide all log output

connection options:

/h, /host: mongodb host to connect to

(setname/host1,host2 for replica sets)

/port: server port (can also use --host hostname:port)

authentication options:

/u, /username: username for authentication

/p, /password: password for authentication

/authenticationDatabase: database that holds the user's credentials

/authenticationMechanism: authentication mechanism to use

storage options:

/d, /db: database to use (default is 'test')

/l, /local: local filename for put|get

/t, /type: content/MIME type for put (optional)

/r, /replace remove other files with same name after put

/prefix: GridFS prefix to use (default is 'fs')

/writeConcern: write concern options e.g. --writeConcern majority,

--writeConcern '{w: 3, wtimeout: 500, fsync: true, j: true}' (defaults to

(2) C:\MongoDB>mongofiles --version

Output:-

mongofiles version: r3.0.12-15-gc0066f6

git version: 550d8c0009702cb8a651e94a0cd6034e276ada84

(3) C:\MongoDB>mongofiles list

Output:-

2016-10-01T14:03:55.385+0530 connected to: localhost

demo.txt 18

(4) C:\MongoDB>mongofiles put demo.txt

Output:-

2016-10-01T14:13:19.914+0530 connected to: localhost

added file: demo.txt

(5) db.fs.files.find().pretty() ---------type this command to the client dos screen

Output:-

{

"_id" : ObjectId("57ef772780769907000000001"),

"chunkSize" : 261120,

"uploadDate" : ISODate("2016-10-01T08:43:19.917Z"),

"length" : 18,

"md5" : "cd0f7e804d7dcc24240ed8d41750faf6",

"filename" : "demo.txt"

}

{

"_id" : ObjectId("57ef77748076990de8000001"),

"chunkSize" : 261120,

"uploadDate" : ISODate("2016-10-01T08:44:36.731Z"),

"length" : 18,

"md5" : "cd0f7e804d7dcc24240ed8d41750faf6",

"filename" : "demo.txt"

}

(6) db.fs.chunks.find() .pretty() ----------type this command to the mongoclient dos screen

Output:-

{

"_id" : ObjectId("57ef772780769907000000002"),

"files_id" : ObjectId("57ef772780769907000000001"),

"n" : 0,

"data" : BinData(0,"aGkNCmhvdyANCmFyZQ0KeW91")

}

{

"_id" : ObjectId("57ef77748076990de8000002"),

"files_id" : ObjectId("57ef77748076990de8000001"),

"n" : 0,

"data" : BinData(0,"aGkNCmhvdyANCmFyZQ0KeW91")

}

(7) C:\MongoDB>mongofiles -d hns delete demo.txt

Output:-

2016-10-03T15:07:16.385+0530 connected to: localhost

successfully deleted all instances of 'demo.txt' from GridFS

Storing Files:-

You use the storeUpload() function to store files into your database with GridFS.

This function takes two parameters:

one indicates the fieldname of the file to be uploaded, and

the other can optionally be used to specify the file's metadata.

Once used, the function reports back the _id of the file stored.

The following simple code example shows how to use the storeUpload() function:

// Connect to the database

$c = new MongoClient();

// Select the name of the database

$db = $c->contacts;

// Define the GridFS class to ensure we can handle the files

$gridFS = $db->getGridFS();

// Specify the HTML field's name attribute

$file = 'fileField';

// Specify the file's metadata (optional)

$metadata = array('uploadDate' => date());

// Upload the file into the database

$id = $gridFS->storeUpload($file,$metadata);

Update Files:-

Note:- Same code as above but just use update() method instead of storeUpload()

```
// Specify the metadata to be added
$metadata = array('$set' => array("Tag" => "Avatar"));
// Specify the search criteria to which to apply the metadata
$criteria = array('_id' => $id);
// Insert the metadata
$db->grid->update($criteria, $metadata);
```

Retrieving Files:-

Of course, the ability to store your files in a database wouldn't do you any good if you weren't able to retrieve these files later.

Retrieving files is as easy as storing them.

The following example shows how to retrieve the filenames stored, which you accomplish using the getFilename() function:

```
// Connect to the database
$c = new MongoClient();
$db = $c->contacts;
// Initialize GridFS
$gridFS = $db->getGridFS();
// Find all files in the GridFS storage and store under the $cursor parameter
$cursor = $gridFS->find();
```

```
// Return all the names of the files

foreach ($cursor as $object) {

echo "Filename:".$object->getFilename();

}
```

Deleting Data:-

You can ensure that any previously stored data is removed by using the delete() function. This function takes one parameter: the _id of the file itself.

The following example illustrates how to use the delete() function to delete the file matching the Object ID 4c555c70be90968001080000:

```
// Connect to the database

$c = new MongoClient();

// Specify the database name

$db = $c->contacts;

// Initialize GridFS

$gridFS = $db->getGridFS();

// Specify the file via it's ID

$id = new MongoId('4c555c70be90968001080000');

$file = $gridFS->findOne(array('_id' => $id));

// Remove the file using the remove() function $gridFS->delete($id);
```

Note:- **https://pymongo.readthedocs.io/en/3.6.0/api/gridfs/index.html**

Q5 Explain database administration in mongodb.

- Mongodb provides different functionality for database administration. This functionality are used to keep the track and control on the database resource.

- The database administration provides the operation and maintains of mongodb instance.

- Database administration provide configuration operation and deployment process and many different task which are as under.

1. Administration

2. Deployment

3. Performance & Monitoring

4. Optimization

5. Replication etc….

Administration:-

- Every database management system support the administration of database.

- It means that hear all the data and different use are managed properly.

- The administration task include new user creation, security, implementation of database model, modulating, backup & recovery of data.

Deployment:-

- It is a one type of installation or implementation process where user allows to write different script code for your data and database.

- Hear the query can be performed in database.

- The indexing, database designing and modules are implemented hear.

Performance & monitoring:-

- Database administration (DBA) must retrieve the performance of database using different tools.

- DBA generate the report for database performance.

- DBA is also responsible for generate and create different user roles and perform basic database administrative task.

Q-6 Explain mongodb database administration basic task.

- In mongodb there are different basic task for database administration which are as under.

Starting and stopping mongodb server & client:-

- Hear you have to start the server first with mongod.exe file.

- You have to start client by using mongo.exe.

- There are different command line option used with mongod command like.

- --help

- --dbpath

- --port

- --config etc…

- If you want to stop mongodb server then you have to use the shutdown command.

- Db.shutdownServer()

File based configuration:-

- Mongodb support reading configuration information from a file the name of this file is mongodb.config.

- The configuration is automatically done by mongodb server.

Monitoring:-

- Hear the mongodb database perform will be checked by monitoring functionality.

- Accessing the data in memory is fast and accessing the date on disk is slow so hear you have to moniter how mongodb interact with disk and memory.

Server state:-

- The server status command is used to provide a static information about mongodb server.

Security:-

- Hear you ensure that your system and data is secure with mongodb server and environment.

- You can setup the firewall also for allow mongodb to connect with internet network and client only.

- Mongodb support authentication like username and password for query connections.

- You can used the command like

- --bind_it

- --nohttpinterface

- --nouniquesocket

- --noscripting

Configuration & Maintenance:

- Hear you can perform different management operation, configuration performance and analysis task.

Login or Logfiles:-

- The log file are used to record all the changes in the database.

- By default mongod send its logs to STD out.

- Most init script use –logpath option to send logs to a file.

Backup & Recovery:-

- It is important to take regular backup of your system so if any failure occurs then you can restore your data back.

- You can use mongodump command for backup purpose and mongorestore command for recovery purpose.

Authentication Task:-

- Each database in mongodb have number of user so it provides security for mongodb communication for reading and writing client and server.

- You can use the command show users and show roles in admin database.

write a note on optimization.

Ans: --> optimization is and act or process to make something is fully perfect.

--> It is a simple method to perform a design or a system as effective as possible.

-->there are many factors which effect the db performance and responsiveness.

-->optimization is used to analyze the db its performance, indexing, query structure, data model, application design, operational factor, architecture, system configuration etc.

-->optimization is used with mongo db including different types of category like.

1) Analyzing mongodb (analyze number of connection, hardware, software etc)

2) To check query performance in mongodb (with db.profiler)

3) To check performance of current operation (with the help of methods)

4) Optimize query performance in mongodb (with indexing)

5) Optimize different design notes with (dynamic schema, dynamic data types, case sensitivity)

1) Analyzing mongodb

-->you can developed and operate the application with mongo db then some time you may need analyze the performance of application and its db.

-->here you need to process for function of db.number of open db connection, hardware, software, configuration etc.

2) To check mongodb performance & 3) check performance of current operation

--> There are different techniques for evaluate the performance of mongo db with different operations.

--> Mongo db provide db profiler that show characteristic of each operation of db.

--> Profiler use to locate different queries for write operation those are running.

--> Profiler provides two different methods which are as under.

a) Db.currentop ()

-->this method is used for display the record of current operation running on mongodb instance.

b) Db.explain ()

-->the explain command provides detail information about a given querys.path in short it return the information stats of query execution.

Example

>use stud

>db.currentop ();

>db.mscit.find ({"rno":1}).explain ("execution stats");

4) Optimize query performance

-->the optimization is very useful for faster processing of multiple queries or a query on multiple fields here it can allow to create the index for searching operations.

-->by using the index structure the query execution will be fast and the index contents very small space in memory.

Syntax

Db. <collname>.createindex ({field name});

Example

db.mscit.createindex ({"name":1});

5) Optimize different design notes

-->during the design notes in optimization technique include different types of data related functionality.

-->it include 3 types of functionality which are as under.

a) Dynamic schema

-->mongodb provides dynamic schema types db and data show there is no need to define the fix structure.

-->due to dynamic schema you can get the benefit of polymorphism and iterative development.

b) Case sensitive string

-->mongo db screen are case sensitive show STUD & stud is different.

c) Data type sensitive field

-->mongo db support different data types as per it value given by the user.

Example

{"rno":1} / /number

{"rno":"1"} / /string

Explain mongodb replication.

Ans :

--> replications the process or a part of backup and recovery concept.

-->it is multiple copies of your data or db which is useful at the time of failover conditions.

-->replication protect the db from lost of data.

-->replication is the process of synchronizing the data across multiple servers.

-->replication allows you to recover the data from hardware failure or some software service damage.

-->replication provides additional copies of data so it can be use for disaster recovery, b'up and reporting.

-->replication provides multiple copies of data on different servers and it will automatically get the updates of changes.

Replication advantage

1) Its keep your data safe.

2) The data can be available at any time.

3) It provides disaster recovery.

4) It cannot recover to down the server for maintains.

5) It provides scaling of data.

6) It is helpful for backup and recovery.

Replication work in mongodb Diagram

-->in mongodb replication is done by using replica set.

-->replica set is group of mongod instance.

-->in replica set one node is primary node and all other nodes are secondary node.

Primary node

-->this node receive all writing operation from other instances.

Secondary node

-->this node contains all other operation from primary node so they have the same data set of primary node.

Rule for replica set

-->replica set can have only one primary node.

-->replica set is a group of two or more nodes.

-->all data replicates from primary to secondary.

-->at the time of failover of primary node automatically another node can selected and that secondary node worked as primary node.

Features of replica set

-->it provides cluster for number of nodes.

-->any one node can be primary and remaining are secondary.

-->all writing operations goes throw primary node.

-->provide automatic failure.

-->provide automatic recovery.

Steps for replication

Step 1: start up a mongo shell with the **--nodb** option, it means that this shell is not connected with any mongod.

(Shutdown the running server and start mongodb server by using replica set command)

Step 2: write the following command.

>mongod--port27017-db path "c:/data" -- replicaset stud-demo

Or if you want to create replica set then write the command

Replicas=new replica set demo ({"nodes":3})

-->it will start mongod instance with stud-demo name

Or

It start replica set demo with one primary and two secondary node.

Q . 9 Explain sharding

Ans:

-->Sharding is the process of storing data records across multiple machines.

-->In mongodb the day by day size of the data increase so a single machine may not be sufficient to store the data.

-->sharding is used to solve the problem with scaling and accept multiple machines to support data growth and reading or writing operation.

Why sharding

-->in replication all writing operation performs in master node.

-->single replica set has limitation of 12 node.

-->memory cannot be large enough when archive data set is big.

-->mongodb use sharding to support deployment with very large data set and reading and writing operation.

-->sharding provides the data set and distribution of data over multiple server (shard).

-->each shard is independent data base.

-->only local disk is not capable to store big data so sharding is require.

Diagram

-->shard are used to store the data in production environment each shard is a separate replica set.

-->in mongodb you can use query router and config server to perform operation and clustering of Meta data.

Sharding characteristic

-->sharding reduce the load.

-->sharding reduce the number of operation.

-->sharding reduce the amount of data for each server needs to store.

Components of sharding

-->there are 3 components of sharding.

1) Shard

2) Query router

3) Config server

1) Shard

-->it is used to store the data.

-->a mongodb shard cluster distributes data across one or more shards.

-->each shard is deploys as a mongodb replica set.

-->the replica set store some part of the total data.

2) query router (mongos router)

-->the query router is also known as mongos instances.

-->each shard contains part of the cluster data then also you need one interface for operation of cluster as a whole.

-->the mongos process is a router that directs all reads and write to the appropriate shard.

-->it is the interface between client application and sharding operation then it returns the results to the client.

-->sharding contents more than one query router to divide client request load.

-->client can send request to one query router.

3) Config server

-->the config server is used to store the cluster meta data.

-->config server to store the shard clusters state.

-->this meta data include the global cluster configuration the location of each database, collection, the particular range of data and the history of the data across the shard.

-->every time mongos process is started then the mongos fetches a copy of the meta data from the config server.

-->you can get an overall view of you cluster by running sh. Status().

-->it will give you a summary of your shards, data base and collections.

Diagram of sharding cluster with 3 config server

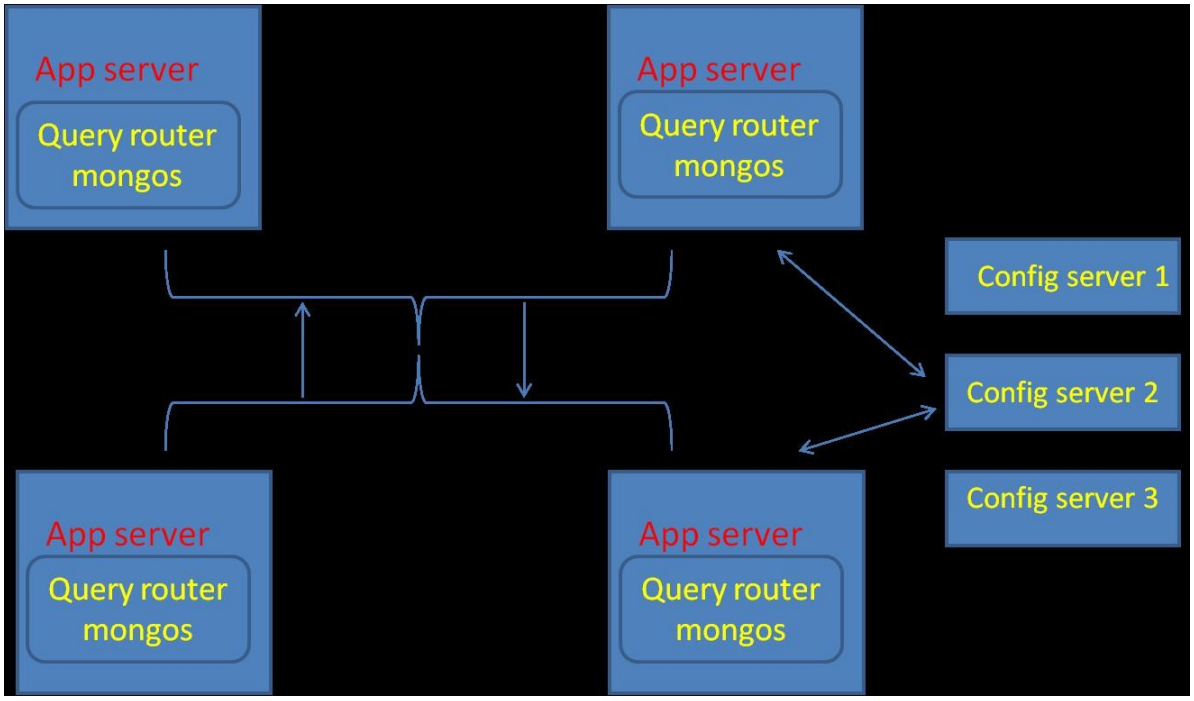
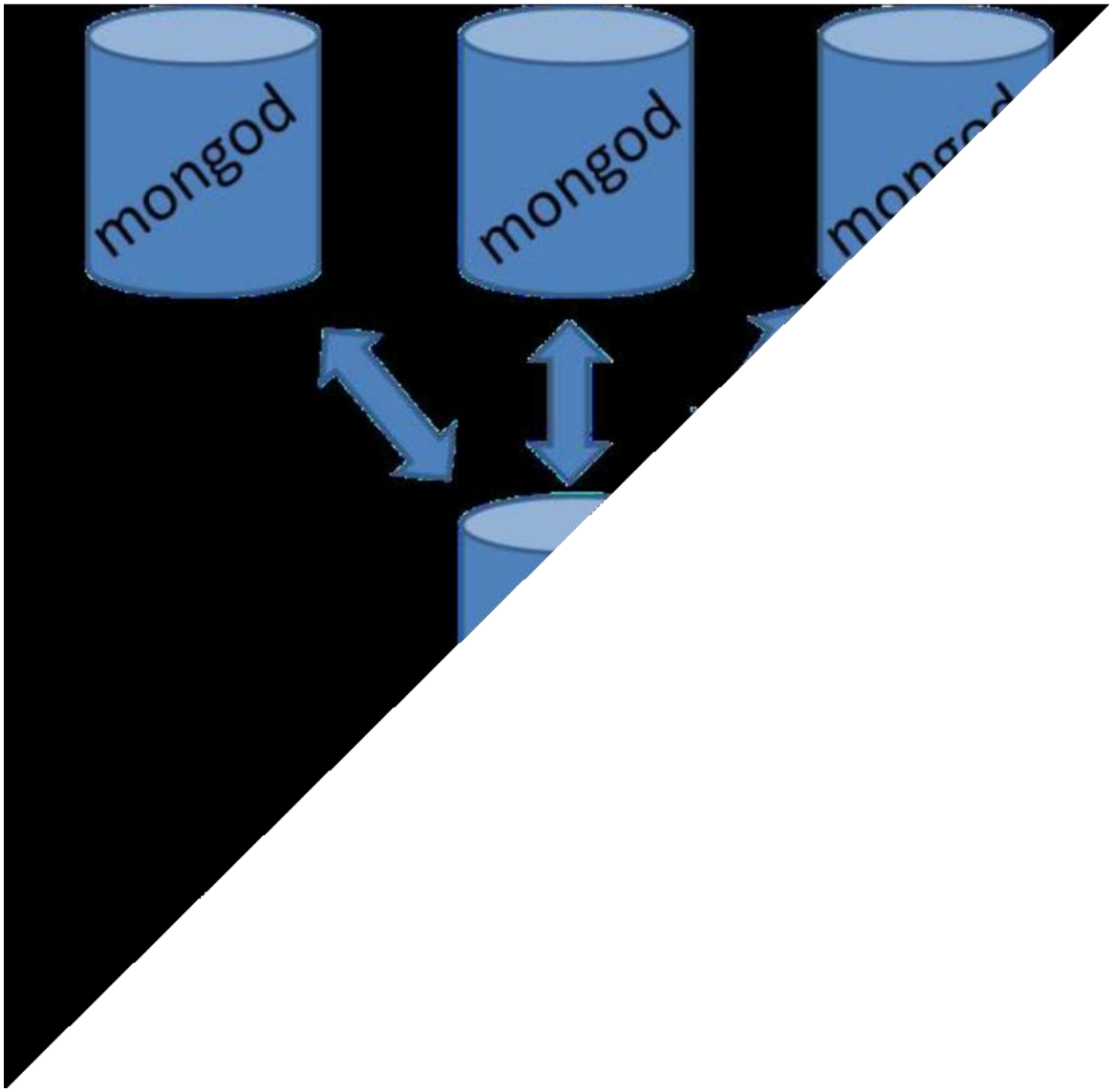


Diagram of sharding Shared client connection



No shared client connection